

COMS20018, Introduction to AI: Neural Networks

Mengyan Zhang

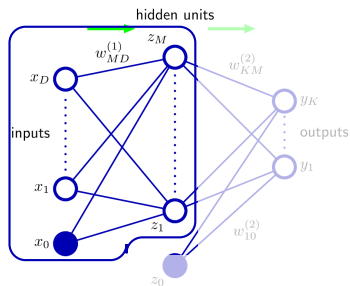
School of Computer Science
University of Bristol

29th September 2026

Agenda

- ▶ Feed-forward network functions: A two-layer network
- ▶ Training a network
 - ▶ A nonconvex loss: global vs. local minima, saddle points
 - ▶ (Mini-batch) SGD
- ▶ Chain rule and error backpropagation
 - ▶ Computing gradients efficiently
 - ▶ Autograd and training loops in PyTorch

A two-layer network (1/3): the hidden layer

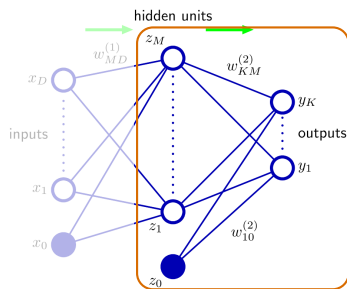


Bishop, *PRML* (2006), Fig. 5.1: filled nodes $x_0=z_0=1$ carry the biases.

- ▶ Input $\mathbf{x} = (x_1, \dots, x_D)^\top$, plus a bias $x_0=1$.
- ▶ Each of the M **hidden units** takes a weighted sum of the inputs, then applies a *differentiable, nonlinear* activation function h :

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i, \quad z_j = h(a_j)$$

A two-layer network (2/3): the output layer



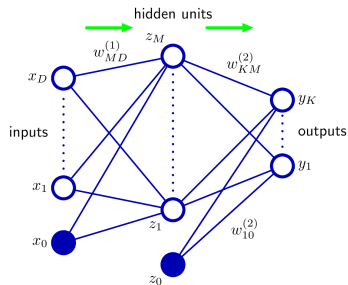
Bishop, *PRML* (2006), Fig. 5.1: filled nodes $x_0=z_0=1$ carry the biases.

- ▶ The hidden units (plus a bias $z_0=1$) feed the outputs $y_1, \dots, y_K$:

$$y_k = f\left(\sum_{j=0}^M w_{kj}^{(2)} z_j\right)$$

- ▶ K outputs: $K=1$ for a scalar target; $K>1$ for a vector target $\mathbf{t} \in \mathbb{R}^K$
- ▶ Output activation f : identity (regression), sigmoid σ (binary classification), softmax (multiclass classification), etc.

A two-layer network (3/3): putting it together

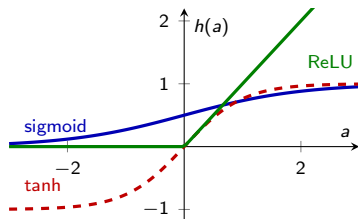


Bishop, *PRML* (2006), Fig. 5.1: filled nodes $x_0=z_0=1$ carry the biases.

- ▶ Substitute the **hidden layer** into the **output layer**.
- ⇒ One differentiable function of *all* weights $\mathbf{w} = (w_{ji}^{(1)}, w_{kj}^{(2)})$; evaluating it left to right is **forward propagation**.

$$y_k(\mathbf{x}, \mathbf{w}) = f\left(\sum_{j=1}^M w_{kj}^{(2)} h\left(\sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right)$$

Activation functions: why the nonlinearity matters



► Common choices for h :

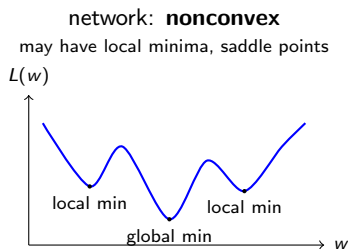
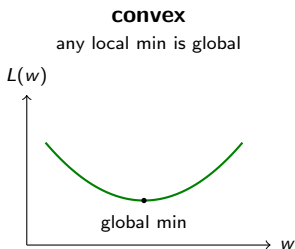
- **sigmoid**, $\sigma(a) = \frac{1}{1 + e^{-a}}$:
output in $(0, 1)$
- **tanh**, $\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}$:
output in $(-1, 1)$, a rescaled sigmoid centred at 0
- **ReLU**, $\max(0, a)$: cheap, its slope doesn't vanish for $a > 0$

⇒ Stack more layers ⇒ a **deep** network:
richer, adaptive basis functions.

- Why nonlinear (for hidden units)? If h were linear, the two layers would collapse into a single linear layer

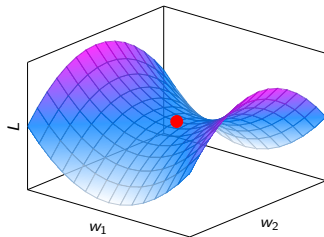
Parameter optimisation

- ▶ Define a loss function over the training set, e.g. sum-of-squares (regression) or cross-entropy (classification), now as a function of *all* the network's weights $\mathbf{w}$.
- ▶ Least squares was *quadratic* (closed form); logistic regression had no closed form, but the loss was convex.
- ▶ For typical multilayer neural networks, $L(\mathbf{w})$ is generally **nonconvex** in $\mathbf{w}$ - No closed-form solution; search for a good $\mathbf{w}$ iteratively



(S)GD on a neural network: what changes?

- ▶ A stationary point, $\nabla L = 0$, can now be a local minimum, the global minimum, or a **saddle point**; different initialisations can lead to different solutions.
 - ▶ In practice: **mini-batch** SGD, averaging ∇L_n over a small batch (e.g. 32–256 examples), balancing noise vs. speed.
- ⇒ Modern refinements: **momentum**, adaptive step sizes (**Adam**); see Goodfellow et al., Ch. 8.



Saddle point: $\nabla L = 0$ (red), yet still downhill in some directions.

Error backpropagation: computing ∇L efficiently

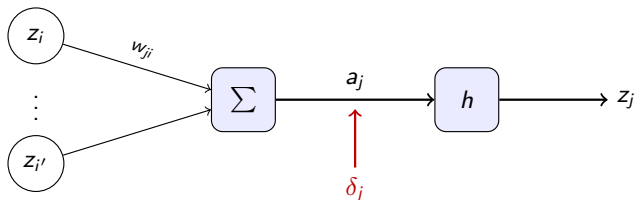
- ▶ A network can easily have $W =$ thousands (or billions) of weights.
- ⇒ **Error backpropagation**: the **chain rule** computes every derivative exactly, at a cost proportional to the number of weights, $O(W)$ (one forward + one backward pass).
- ▶ Losses from maximum likelihood with i.i.d. data are a **sum over data points**:

$$L(\mathbf{w}) = \sum_{n=1}^N L_n(\mathbf{w})$$

- ▶ So we only need ∇L_n for **one data point** n : use it directly for SGD, or sum over n for batch gradient descent.

Following Bishop, *PRML* (2006), §5.3.1

A general network: error signals δ

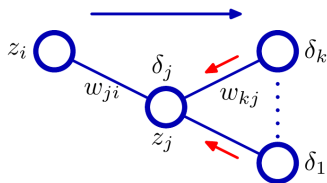


- ▶ Each unit: $a_j = \sum_i w_{ji} z_i$, $z_j = h(a_j)$, where z_i is an input or another unit (bias: $z = 1$). Applying these in turn is **forward propagation**.
- ▶ L_n depends on w_{ji} only via a_j , so by the chain rule, with the **error** $\delta_j \equiv \partial L_n / \partial a_j$ and $\partial a_j / \partial w_{ji} = z_i$:

$$\frac{\partial L_n}{\partial w_{ji}} = \frac{\partial L_n}{\partial a_j} \frac{\partial a_j}{\partial w_{ji}} = \delta_j z_i$$

δ at the output end of the link $\times z$ at its input end.

Hidden units: backpropagating the δ 's



Bishop, *PRML* (2006), Fig. 5.7: blue, forward propagation; red, backward propagation of the errors.

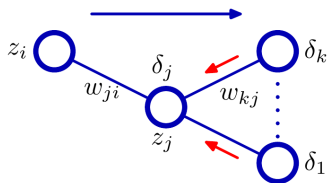
- ▶ Unit j affects L_n only via the units k it sends connections to, so sum over them:

$$\delta_j \equiv \frac{\partial L_n}{\partial a_j} = \sum_k \frac{\partial L_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

- ▶ Using $a_k = \sum_j w_{kj} z_j$ and $z_j = h(a_j)$, the **backpropagation formula**:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

Hidden units: backpropagating the δ 's



Bishop, *PRML* (2006), Fig. 5.7: blue, forward propagation; red, backward propagation of the errors.

- ▶ Unit j affects L_n only via the units k it sends connections to, so sum over them:

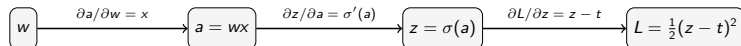
$$\delta_j \equiv \frac{\partial L_n}{\partial a_j} = \sum_k \frac{\partial L_n}{\partial a_k} \frac{\partial a_k}{\partial a_j}$$

- ▶ Using $a_k = \sum_j w_{kj} z_j$ and $z_j = h(a_j)$, the **backpropagation formula**:

$$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$$

- ▶ For the **output** units, e.g. sum-of-squares loss with identity output: $\delta_k = y_k - t_k$.
- ⇒ Start from the outputs' δ_k and apply this recursively: every hidden unit's δ , for any feed-forward network.
- ⇒ **Key idea**: compute δ once at each unit, then reuse it for all of that unit's incoming weights.

Try it: the chain rule

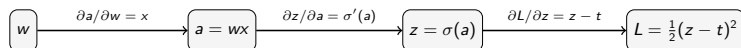


- **Chain rule:** multiply the local derivatives along the path,
 $\partial L / \partial w = (\partial L / \partial z) (\partial z / \partial a) (\partial a / \partial w)$.

Try it (2 min)

With $w = 0$, $x = 2$, $t = 1$: compute a , z , L , then $\partial L / \partial w$. (Hint: $\sigma'(a) = \sigma(a)(1 - \sigma(a))$.)

Try it: the chain rule



- **Chain rule:** multiply the local derivatives along the path,
 $\partial L / \partial w = (\partial L / \partial z) (\partial z / \partial a) (\partial a / \partial w)$.

Try it (2 min)

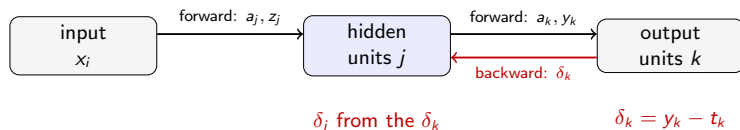
With $w = 0$, $x = 2$, $t = 1$: compute a , z , L , then $\partial L / \partial w$. (Hint: $\sigma'(a) = \sigma(a)(1 - \sigma(a))$.)

$a = 0$, $z = \sigma(0) = 0.5$, $L = 0.125$, and

$$\frac{\partial L}{\partial w} = (z - t) \sigma'(a) x = (-0.5)(0.25)(2) = -0.25$$

negative, so increasing w lowers L .

Backpropagation: the algorithm



Error backpropagation, for each data point $(\mathbf{x}_n, \mathbf{t}_n)$

- 1. Forward propagate** $a_j = \sum_i w_{ji} z_i, \quad z_j = h(a_j)$ every unit
- 2. Output errors** $\delta_k = y_k - t_k$ squared loss + identity output
- 3. Backpropagate** $\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$ deeper nets: repeat layer by layer
- 4. Derivatives** $\partial L_n / \partial w_{ji} = \delta_j z_i$ every weight

- Batch methods: sum over data points, $\partial L / \partial w_{ji} = \sum_n \partial L_n / \partial w_{ji}$.
- ⇒ Then take a (S)GD step; this *is* how every network in this unit is trained.

Backpropagation in PyTorch: autograd

- ▶ In practice we do not need to code the backward pass by hand: frameworks such as **PyTorch** record the forward computation and apply the chain rule automatically (**autograd**).
- ▶ The worked example, in a few lines:

```
import torch
w1 = torch.tensor(0.5, requires_grad=True)  # track gradients
w2 = torch.tensor(3.0, requires_grad=True)
x, t = torch.tensor(2.0), torch.tensor(1.0)

y = w2 * torch.relu(w1 * x)                # forward pass
loss = 0.5 * (y - t) ** 2                  # L_n = 2
loss.backward()                             # backpropagation
print(w1.grad, w2.grad)                    # tensor(12.) tensor(2.)
```

Training a network in PyTorch

```
from torch import nn
model = nn.Sequential(nn.Linear(D, M), nn.ReLU(),      # hidden layer, h = ReLU
                      nn.Linear(M, K))                # output layer, f = identity
loss_fn = nn.MSELoss()                                # mean squared error
# classification: nn.BCEWithLogitsLoss(), which applies the sigmoid itself
opt = torch.optim.SGD(model.parameters(), lr=0.01)    # eta = 0.01

for epoch in range(n_epochs):                          # repeat for several epochs
    for xb, tb in loader:                              # mini-batches, shuffled each epoch
        y = model(xb)                                  # forward pass
        loss = loss_fn(y, tb)
        opt.zero_grad()                                # clear old gradients
        loss.backward()                                # backpropagation: all dL/dw
        opt.step()                                     # SGD update: w ← w - eta * grad
```

- Sizes `D`, `M`, `K`, the mini-batch loader and `n_epochs` are set up beforehand.
- ⇒ The `model`, the `loss` and the optimiser can change (e.g. `torch.optim.Adam`).

Reading

- ▶ Bishop, C. M., *Pattern Recognition and Machine Learning* (2006), Chapter 5: §5.1 (feed-forward network functions), §5.2 (network training, §5.2.4 gradient descent), §5.3 (error backpropagation). [Free online](#).
- ▶ Goodfellow, Bengio, Courville, *Deep Learning* (2016), Chapters 6 and 8, a more modern treatment (ReLU, Adam, and optimisation for deep nets). [Free online](#) (optional, further reading).